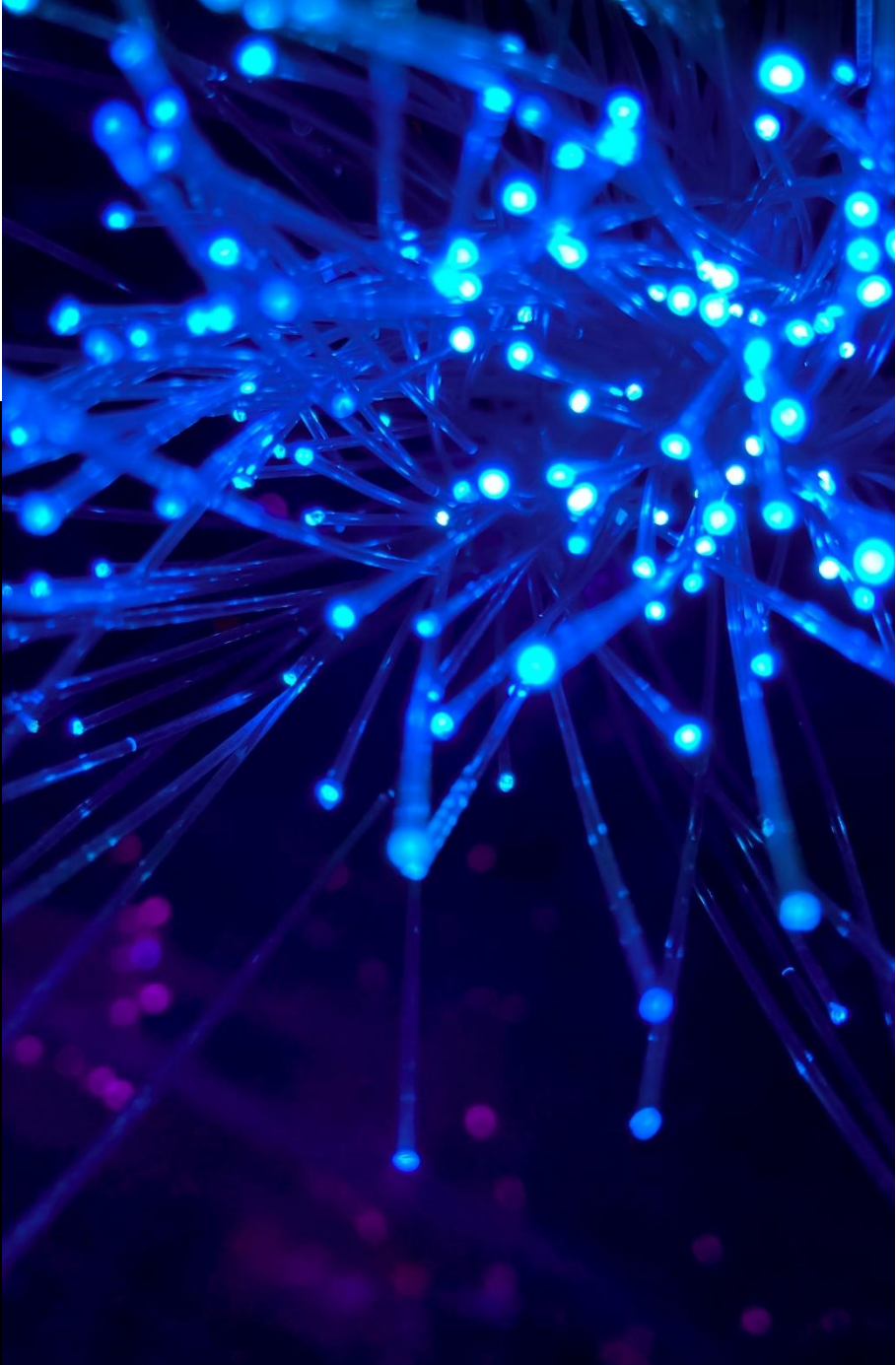




Gestión de Datos

Almacenamiento de datos I: SQL



SQL: fundamentos

¿Qué es SQL?

- SQL significa lenguaje de consulta estructurado
- SQL permite acceder a y manipular bases de datos

¿Qué puede hacer SQL?

- ejecutar consultas contra una base de datos
- recuperar datos de una base de datos
- insertar registros en una base de datos
- actualizar registros en una base de datos
- eliminar registros de una base de datos
- crear nuevas bases de datos
- crear nuevas tablas en una base de datos
- crear procedimientos almacenados en una base de datos
- crear vistas en una base de datos
- establecer permisos en tablas, procedimientos y vistas

Aunque SQL es un estándar, existen diferentes versiones del lenguaje SQL. Sin embargo, todos admiten al menos los comandos principales (como **SELECT**, **UPDATE**, **DELETE**, **INSERT**, **WHERE**) de manera similar.

Algunos comandos SQL importantes

- **SELECT**- extrae datos de una base de datos
- **UPDATE**- actualiza datos en una base de datos
- **DELETE**- elimina datos de una base de datos
- **INSERT INTO**- inserta nuevos datos en una base de datos
- **CREATE DATABASE**- crea una nueva base de datos
- **ALTER DATABASE**- modifica una base de datos
- **CREATE TABLE**- crea una nueva tabla
- **ALTER TABLE**- modifica una tabla
- **DROP TABLE**- elimina una tabla
- **CREATE INDEX**- crea un índice (clave de búsqueda)
- **DROP INDEX**- elimina un índice

Principales Categorías de Datos en SQL:

■ Datos Numéricos:

- Enteros:
 - TINYINT (1 byte)
 - SMALLINT (2 bytes)
 - INT/INTEGER (4 bytes)
 - BIGINT (8 bytes)
- Exactos: DECIMAL
- Aproximados: FLOAT

■ Datos de Cadena (Texto):

- CHAR(n): Longitud fija de caracteres.
- VARCHAR(n): Longitud variable de hasta n caracteres.
- TEXT/NTEXT: Para textos grandes.

■ Datos de Fecha y Hora:

- DATE: Almacena años, meses y días.
- TIME: Almacena horas, minutos y segundos.
- DATETIME / TIMESTAMP: Combina fecha y hora.

■ Datos Binarios:

- BINARY/VARBINARY: Datos binarios de longitud fija/variable.
- BLOB (Binary Large Object): Para imágenes o archivos.

■ Otros Tipos:

- BOOLEAN / BIT: Valores verdadero o falso.

SQL, or *Structured Query Language*, is a language to talk to databases. It allows you to select specific data and to build complex reports. Today, SQL is a universal language of data. It is used in practically all technologies that process data.

SAMPLE DATA

COUNTRY				
id	name	population	area	
1	France	66600000	640680	
2	Germany	80700000	357000	
...	...	...	...	

CITY				
id	name	country_id	population	rating
1	Paris	1	2243000	5
2	Berlin	2	3460000	3
...	...	...	...	...

QUERYING SINGLE TABLE

Fetch all columns from the country table:

```
SELECT *
FROM country;
```

Fetch id and name columns from the city table:

```
SELECT id, name
FROM city;
```

Fetch city names sorted by the rating column in the default ASCending order:

```
SELECT name
FROM city
ORDER BY rating [ASC];
```

Fetch city names sorted by the rating column in the DESCending order:

```
SELECT name
FROM city
ORDER BY rating DESC;
```

ALIASES

COLUMNS

```
SELECT name AS city_name
FROM city;
```

TABLES

```
SELECT co.name, ci.name
FROM city AS ci
JOIN country AS co
ON ci.country_id = co.id;
```

FILTERING THE OUTPUT

COMPARISON OPERATORS

Fetch names of cities that have a rating above 3:

```
SELECT name
FROM city
WHERE rating > 3;
```

Fetch names of cities that are neither Berlin nor Madrid:

```
SELECT name
FROM city
WHERE name != 'Berlin'
AND name != 'Madrid';
```

TEXT OPERATORS

Fetch names of cities that start with a 'P' or end with an 's':

```
SELECT name
FROM city
WHERE name LIKE 'P%'
OR name LIKE '%s';
```

Fetch names of cities that start with any letter followed by 'ublin' (like Dublin in Ireland or Lublin in Poland):

```
SELECT name
FROM city
WHERE name LIKE '_ublin';
```

OTHER OPERATORS

Fetch names of cities that have a population between 500K and 5M:

```
SELECT name
FROM city
WHERE population BETWEEN 500000 AND 5000000;
```

Fetch names of cities that don't miss a rating value:

```
SELECT name
FROM city
WHERE rating IS NOT NULL;
```

Fetch names of cities that are in countries with IDs 1, 4, 7, or 8:

```
SELECT name
FROM city
WHERE country_id IN (1, 4, 7, 8);
```

QUERYING MULTIPLE TABLES

INNER JOIN

JOIN (or explicitly **INNER JOIN**) returns rows that have matching values in both tables.

```
SELECT city.name, country.name
FROM city
[INNER] JOIN country
ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
3	Warsaw	4	3	Iceland

LEFT JOIN

LEFT JOIN returns all rows from the left table with corresponding rows from the right table. If there's no matching row, **NULLs** are returned as values from the second table.

```
SELECT city.name, country.name
FROM city
LEFT JOIN country
ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
3	Warsaw	4	NULL	NULL

RIGHT JOIN

RIGHT JOIN returns all rows from the right table with corresponding rows from the left table. If there's no matching row, **NULLs** are returned as values from the left table.

```
SELECT city.name, country.name
FROM city
RIGHT JOIN country
ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
NULL	NULL	NULL	3	Iceland

SQL Basics Cheat Sheet

FULL JOIN

FULL JOIN (or explicitly **FULL OUTER JOIN**) returns all rows from both tables – if there's no matching row in the second table, **NULLs** are returned.

```
SELECT city.name, country.name
FROM city
FULL [OUTER] JOIN country
ON city.country_id = country.id;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
2	Berlin	2	2	Germany
3	Warsaw	4	NULL	NULL
NULL	NULL	NULL	3	Iceland

CROSS JOIN

CROSS JOIN returns all possible combinations of rows from both tables. There are two syntaxes available.

```
SELECT city.name, country.name
FROM city
CROSS JOIN country;
```

```
SELECT city.name, country.name
FROM city, country;
```

CITY			COUNTRY	
id	name	country_id	id	name
1	Paris	1	1	France
1	Paris	1	2	Germany
2	Berlin	2	1	France
2	Berlin	2	2	Germany

NATURAL JOIN

NATURAL JOIN will join tables by all columns with the same name.

```
SELECT city.name, country.name
FROM city
NATURAL JOIN country;
```

CITY			COUNTRY	
country_id	id	name	name	id
6	6	San Marino	San Marino	6
7	7	Vatican City	Vatican City	7
5	9	Greece	Greece	9
10	11	Monaco	Monaco	10

NATURAL JOIN used these columns to match rows: **city.id, city.name, country.id, country.name**. **NATURAL JOIN** is very rarely used in practice.

AGGREGATION AND GROUPING

GROUP BY **groups** together rows that have the same values in specified columns. It computes summaries (aggregates) for each unique combination of values.

CITY				
id	name	country_id		
1	Paris	1		
101	Marseille	1		
102	Lyon	1		
2	Berlin	2		
103	Hamburg	2		
104	Munich	2		
3	Warsaw	4		
105	Cracow	4		

→

CITY			
country_id	count		
1	3		
2	3		
4	2		

AGGREGATE FUNCTIONS

- avg(expr) – average value for rows within the group
- count(expr) – count of values for rows within the group
- max(expr) – maximum value within the group
- min(expr) – minimum value within the group
- sum(expr) – sum of values within the group

EXAMPLE QUERIES

Find out the number of cities:

```
SELECT COUNT(*)
FROM city;
```

Find out the number of cities with non-null ratings:

```
SELECT COUNT(rating)
FROM city;
```

Find out the number of distinctive country values:

```
SELECT COUNT(DISTINCT country_id)
FROM city;
```

Find out the smallest and the greatest country populations:

```
SELECT MIN(population), MAX(population)
FROM country;
```

Find out the total population of cities in respective countries:

```
SELECT country_id, SUM(population)
FROM city
GROUP BY country_id;
```

Find out the average rating for cities in respective countries if the average is above 3.0:

```
SELECT country_id, AVG(rating)
FROM city
GROUP BY country_id
HAVING AVG(rating) > 3.0;
```

SUBQUERIES

A subquery is a query that is nested inside another query, or inside another subquery. There are different types of subqueries.

SINGLE VALUE

The simplest subquery returns exactly one column and exactly one row. It can be used with comparison operators =, <, <=, >, or >=.

This query finds cities with the same rating as Paris:

```
SELECT name
FROM city
WHERE rating = (
    SELECT rating
    FROM city
    WHERE name = 'Paris'
);
```

MULTIPLE VALUES

A subquery can also return multiple columns or multiple rows. Such subqueries can be used with operators IN, EXISTS, ALL, or ANY.

This query finds cities in countries that have a population above 20M:

```
SELECT name
FROM city
WHERE country_id IN (
    SELECT country_id
    FROM country
    WHERE population > 20000000
);
```

CORRELATED

A correlated subquery refers to the tables introduced in the outer query. A correlated subquery depends on the outer query. It cannot be run independently from the outer query.

This query finds cities with a population greater than the average population in the country:

```
SELECT *
FROM city main_city
WHERE population > (
    SELECT AVG(population)
    FROM city average_city
    WHERE average_city.country_id = main_city.country_id
);
```

This query finds countries that have at least one city:

```
SELECT name
FROM country
WHERE EXISTS (
    SELECT *
    FROM city
    WHERE country_id = country.id
);
```

SET OPERATIONS

Set operations are used to combine the results of two or more queries into a single result. The combined queries must return the same number of columns and compatible data types. The names of the corresponding columns can be different.

CYCLING			SKATING		
id	name	country	id	name	country
1	YK	DE	1	YK	DE
2	ZG	DE	2	DF	DE
3	WT	PL	3	AK	PL
...	...	...	...	...	...

UNION

UNION combines the results of two result sets and removes duplicates. **UNION ALL** doesn't remove duplicate rows.

This query displays German cyclists together with German skaters:

```
SELECT name
FROM cycling
WHERE country = 'DE'
UNION / UNION ALL
SELECT name
FROM skating
WHERE country = 'DE';
```



INTERSECT

INTERSECT returns only rows that appear in both result sets.

This query displays German cyclists who are also German skaters at the same time:

```
SELECT name
FROM cycling
WHERE country = 'DE'
INTERSECT
SELECT name
FROM skating
WHERE country = 'DE';
```



EXCEPT

EXCEPT returns only the rows that appear in the first result set but do not appear in the second result set.

This query displays German cyclists unless they are also German skaters at the same time:

```
SELECT name
FROM cycling
WHERE country = 'DE'
EXCEPT / MINUS
SELECT name
FROM skating
WHERE country = 'DE';
```





Selection Queries

Clause	How to use	Explained
SELECT	<pre>SELECT * FROM products;</pre>	Display all columns from <code>products</code> table.
	<pre>SELECT productName, buyPrice FROM products;</pre>	Display only <code>productName</code> and <code>buyPrice</code> columns from <code>products</code> table.
ORDER BY	<pre>SELECT productName, buyPrice FROM products ORDER BY buyPrice DESC;</pre>	Sort the selected columns by <code>buyPrice</code> in descending order.
	<pre>SELECT productName, buyPrice FROM products ORDER BY productName ASC;</pre>	Sort the selected columns by <code>productName</code> in ascending order.
	<pre>SELECT orderNumber, customerNumber, orderDate, status FROM orders ORDER BY customerNumber ASC, orderDate DESC;</pre>	Sorts the data by <code>customerNumber</code> and then by <code>orderDate</code> within each <code>customerNumber</code> .
DISTINCT	<pre>SELECT DISTINCT productLine FROM products;</pre>	Retrieve unique values from <code>productLine</code> in <code>products</code> table.
	<pre>SELECT DISTINCT city, country FROM customers ORDER BY country, city;</pre>	Retrieve unique combinations of <code>city</code> and <code>country</code> where <code>customers</code> are located, sorted by <code>country</code> and then <code>city</code> .

SQL Aliases

SQL aliases are used to give a column or a table a temporary name.

- Aliases are used to make column names more readable.
- An alias only exists for the duration of that query.
- An alias is created with the **AS** keyword.

Alias for column:

```
SELECT column_name AS alias_name  
FROM table_name;
```

Alias for table:

```
SELECT column_name(s)  
FROM table_name AS alias_name;
```



Aggregate Functions

Clause	How to use	Explained
SUM	<pre>SELECT SUM(quantityOrdered * priceEach) AS total_sales FROM orderdetails;</pre>	Calculates the total sales from the <code>orderdetails</code> table.
AVG	<pre>SELECT AVG(buyPrice) AS average_price FROM products;</pre>	Averages the <code>buyPrice</code> values in <code>products</code> .
ROUND	<pre>SELECT ROUND(AVG(buyPrice), 2) AS average_price FROM products;</pre>	Rounds the average of <code>buyPrice</code> to two decimal places.
MIN	<pre>SELECT MIN(buyPrice) AS lowest_price FROM products;</pre>	Finds the minimum value in the <code>buyPrice</code> column of <code>products</code> .
MAX	<pre>SELECT MAX(buyPrice) AS highest_price FROM products;</pre>	Finds the maximum value in the <code>buyPrice</code> column of <code>products</code> .
COUNT	<pre>SELECT COUNT(*) AS total_orders FROM orders;</pre>	Counts the total number of rows in <code>orders</code> .

Note COUNT(*) includes all rows, while COUNT(column_name) excludes NULL values in the specified column.



Aggregate Functions

Clause

How to use

Explained

HAVING

```
SELECT productLine, AVG(buyPrice)
      AS avg_price
FROM products
GROUP BY productLine
HAVING AVG(buyPrice) > 50;
```

Filters product lines to only include those with average price greater than 50.

COUNT

```
SELECT COUNT(*) AS total_products
FROM products;
```

Counts the total number of rows in the products table, returning the total number of products. This includes all rows, regardless of NULL values in any columns.

```
SELECT COUNT(reportsTo)
      AS employees_with_manager
FROM employees;
```

Counts the number of non-null values in the reportsTo column of the employees table, showing how many employees have a manager assigned.

COUNT ignores **NULL** values, so employees without a manager (e.g., the president) are not included in this count.



Aggregate Functions

Clause

How to use

Explained

GROUP BY

```
SELECT productLine, AVG(buyPrice)
      AS avg_price
FROM products
GROUP BY productLine;
```

Groups rows by `productLine` and calculates the average price for each product line.

```
SELECT productLine, AVG(buyPrice)
      AS avg_price
FROM products
WHERE buyPrice > 100
GROUP BY productLine;
```

Groups rows by `productLine` for products with price over 100 and calculates the average price for each product line.

```
SELECT customerNumber, COUNT(orderNumber)
      AS order_count
FROM orders
WHERE orderDate >= '2023-01-01'
GROUP BY customerNumber
ORDER BY order_count DESC;
```

Groups orders by `customerNumber`, counts the number of orders for each customer in 2023 and beyond, and sorts the results by the order count in descending order.

This shows which customers placed the most orders in 2023.

String Functions

Clause	How to use	Explained
UPPER	<pre>SELECT UPPER(productName) AS uppercase_name FROM products;</pre>	Converts the <code>productName</code> column values to uppercase.
LOWER	<pre>SELECT LOWER(productName) AS lowercase_name FROM products;</pre>	Converts the <code>productName</code> column values to lowercase.
LENGTH	<pre>SELECT productName, LENGTH(productName) AS name_length FROM products;</pre>	Calculates the length of each value in the <code>productName</code> column.
SUBSTR	<pre>SELECT SUBSTR(productLine, 1, 3) AS product_category, productLine FROM products;</pre>	Extracts the first three characters from the <code>productLine</code> column. <code>SUBSTR</code> extracts a substring from a given string. It can be used to extract characters from the beginning, end, or any position within the string.

String Functions

Clause	How to use	Explained
	<pre>SELECT SUBSTR(productCode, -4) AS product_id, productCode FROM products;</pre>	Extracts the last four characters from the <code>productCode</code> column.
CONCAT USING	<pre>SELECT firstName ' ' lastName AS full_name FROM employees;</pre>	Concatenates <code>firstName</code> and <code>lastName</code> with a space in between.
	<pre>SELECT firstName '.' lastName '@classicmodelcars.com' AS email_address FROM employees;</pre>	Creates an email address by concatenating <code>firstName</code> , <code>lastName</code> , and domain.



Conditional Queries

Clause

How to use

Explained

CASE

```
SELECT productName,
       buyPrice,
       CASE
         WHEN buyPrice < 50 THEN
           'Budget'
         WHEN buyPrice BETWEEN 50
          AND 100 THEN 'Mid-range'
         ELSE 'Premium'
       END AS price_category
FROM products;
```

Categorizes the `buyPrice` values into Budget, Mid-range, and Premium categories.

```
SELECT orderNumber,
       orderDate,
       CASE
         WHEN CAST(strftime('%m',
          orderDate) AS INTEGER)
          BETWEEN 3 AND 5 THEN
           'Spring Sale'

         WHEN CAST(strftime('%m',
          orderDate) AS INTEGER)
          BETWEEN 6 AND 8 THEN
           'Summer Sale'
         WHEN CAST(strftime('%m',
          orderDate) AS INTEGER)
          BETWEEN 9 AND 11 THEN
           'Fall Sale'
         ELSE 'Winter Sale'
       END AS sale_season
FROM orders;
```

Categorizes orders into different sale seasons based on the order date.

Clause

How to use

Explained

COALESCE

```
SELECT productName,
       COALESCE(productDescription,
        'No description available')
       AS product_description
FROM products;
```

Returns 'No description available' if `productDescription` is `null`.

```
SELECT employeeNumber,
       firstName,
       lastName,
       COALESCE(extension, email, 'No
        contact information') AS contact_info
FROM employees;
```

Returns the first non-null value among extension, email, or 'No contact information'.

CAST

```
SELECT orderNumber, CAST(orderDate AS DATE)
       AS order_day
FROM orders;
```

Converts the `orderDate` to `DATE` type.

Combine Data

Clause	How to use	Explained
UNION	<pre>SELECT productName FROM products WHERE productLine = 'Classic Cars' UNION SELECT productName FROM products WHERE productLine = 'Vintage Cars';</pre>	Combines the product names from 'Classic Cars' and 'Vintage Cars' product lines, removing duplicates.
UNION ALL	<pre>SELECT productName FROM products WHERE productLine = 'Classic Cars' UNION ALL SELECT productName FROM products WHERE productLine = 'Vintage Cars';</pre>	Combines the product names from 'Classic Cars' and 'Vintage Cars' product lines without removing duplicates.
EXCEPT	<pre>SELECT productCode, productName FROM products EXCEPT SELECT productCode, productName FROM products WHERE productLine = 'Classic Cars';</pre>	Returns products EXCEPT the 'Classic Cars' product line, demonstrating how EXCEPT removes rows from the first result that appear in the second result.

Clause	How to use	Explained
INTERSECT	<pre>SELECT customerNumber, customerName FROM customers WHERE country = 'USA' INTERSECT SELECT customerNumber, customerName FROM customers WHERE creditLimit > 100000;</pre>	Returns customers who are both located in the USA and have a credit limit over 100,000. This query demonstrates how INTERSECT finds common rows between two result sets.

Note EXCEPT and INTERSECT are not supported in all SQL databases. These examples use PostgreSQL syntax.

Window Functions

Note SQLite does not support window functions natively. The following examples use PostgreSQL syntax and require PostgreSQL or a SQLite extension.

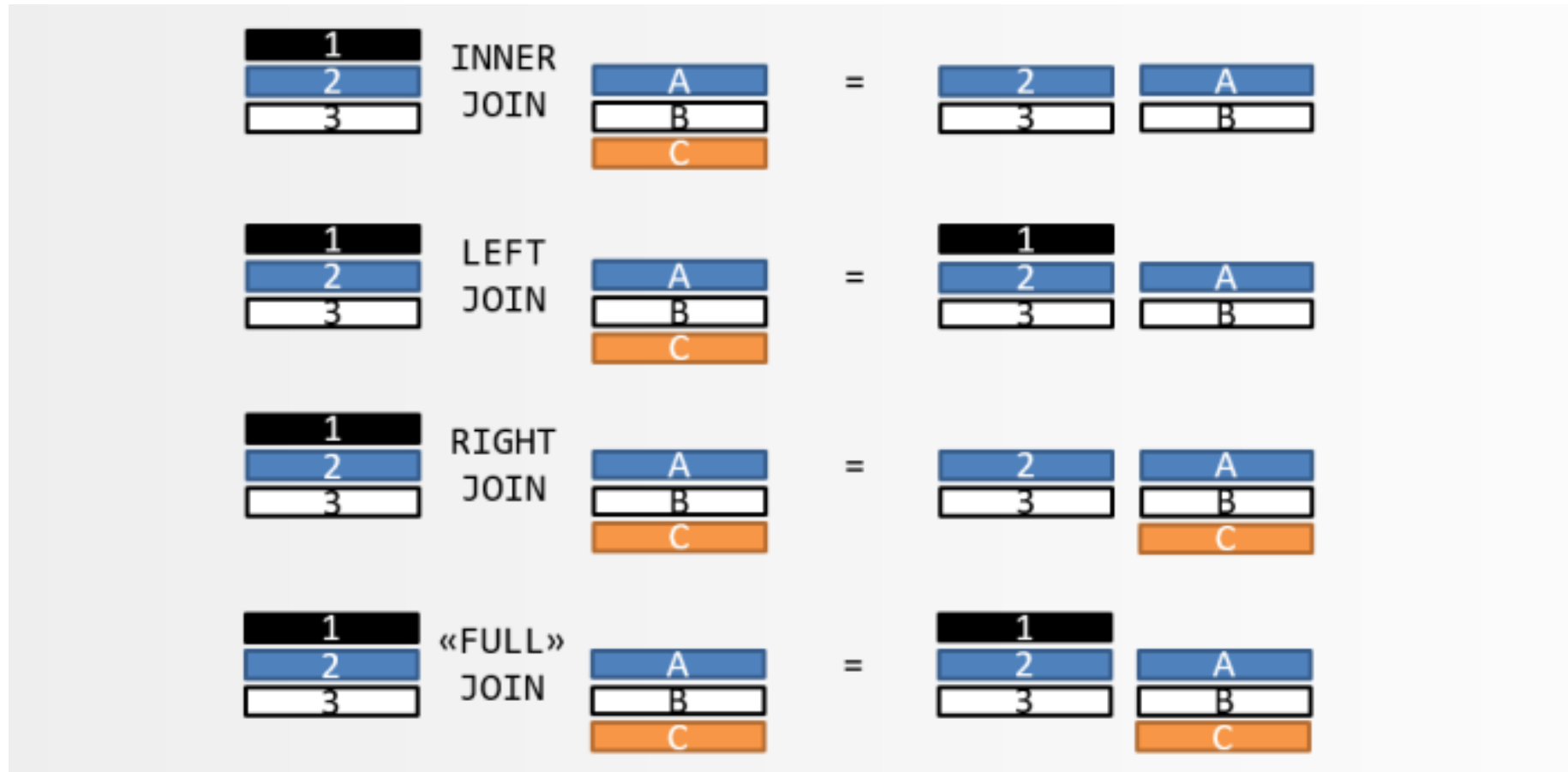
Clause	How to use	Explained
PARTITION BY	<pre>SELECT employeeNumber, officeCode, extension, AVG(LENGTH(extension)) OVER (PARTITION BY officeCode) AS avg_extension_length FROM employees;</pre>	Calculates the average extension length within each office. The PARTITION BY clause divides the data into partitions based on the officeCode column.
ORDER BY	<pre>SELECT employeeNumber, officeCode, extension, SUM(LENGTH(extension)) OVER (ORDER BY LENGTH(extension) DESC) AS running_total_length FROM employees;</pre>	Calculates a running total of extension lengths ordered by length in descending order.
PARTITION BY ORDER BY	<pre>SELECT employeeNumber, officeCode, extension, SUM(LENGTH(extension)) OVER (PARTITION BY officeCode ORDER BY LENGTH(extension) DESC) AS running_total_length FROM employees;</pre>	Calculates a running total of extension lengths within each office, ordered by length.

Ranking Functions

Note SQLite does not support ranking functions natively. The following examples use PostgreSQL syntax and require PostgreSQL or a SQLite extension.

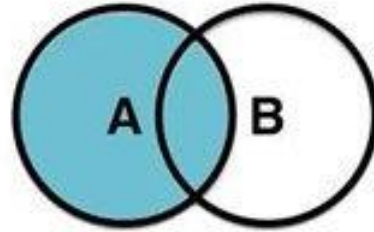
Clause	How to use	Explained
DENSE RANK	<pre>SELECT productCode, productName, buyPrice, DENSE_RANK() OVER (ORDER BY buyPrice DESC) AS price_rank FROM products;</pre>	Ranks products based on <code>buyPrice</code> in descending order. Differs from <code>RANK</code> by handling ties differently (no gaps in ranking).
RANK	<pre>SELECT employeeNumber, officeCode, extension, RANK() OVER (PARTITION BY officeCode ORDER BY LENGTH(extension) DESC) AS extension_rank_in_office FROM employees;</pre>	Ranks employees within each office based on their extension length. Differs from <code>DENSE_RANK</code> by leaving gaps in ranking when there are ties.
ROW NUMBER	<pre>SELECT orderNumber, orderDate, customerNumber, ROW_NUMBER() OVER (ORDER BY orderDate, customerNumber) AS order_number FROM orders;</pre>	Assigns a unique row number to each order based on <code>orderDate</code> and <code>customerNumber</code> .

SQL JOINS

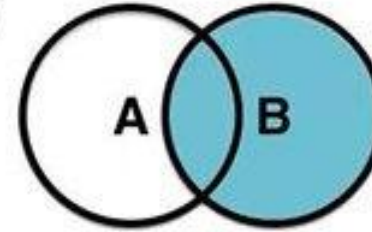


Say NO to Venn Diagrams When Explaining JOINS

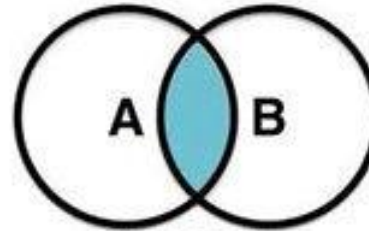
SQL JOINS



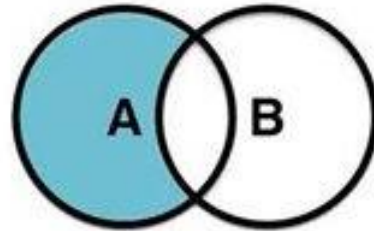
```
SELECT <fields list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key
```



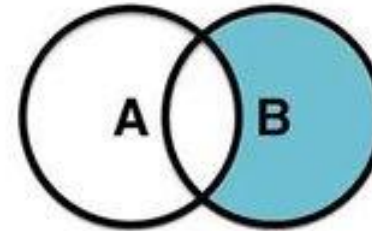
```
SELECT <fields list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key
```



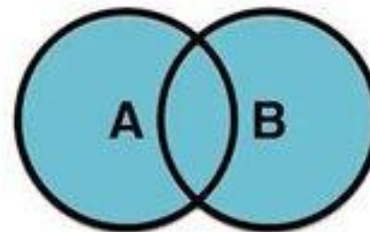
```
SELECT <fields list>  
FROM TableA A  
INNER JOIN TableB B  
ON A.Key = B.Key
```



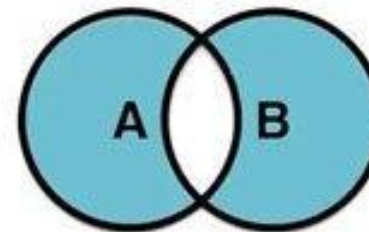
```
SELECT <fields list>  
FROM TableA A  
LEFT JOIN TableB B  
ON A.Key = B.Key  
WHERE B.Key IS NULL
```



```
SELECT <fields list>  
FROM TableA A  
RIGHT JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL
```



```
SELECT <fields list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key
```



```
SELECT <fields list>  
FROM TableA A  
FULL OUTER JOIN TableB B  
ON A.Key = B.Key  
WHERE A.Key IS NULL OR B.Key IS NULL
```

Joins

Clause	How to use	Explained
INNER JOIN	<pre>SELECT o.orderNumber, o.orderDate, c.customerName FROM orders AS o INNER JOIN customers AS c ON o.customerNumber = c.customerNumber;</pre>	Joins <code>orders</code> and <code>customers</code> tables, returning only matching rows. This is the default join type when <code>JOIN</code> is used without specifying <code>LEFT</code> , <code>RIGHT</code> , or <code>FULL</code> .
LEFT JOIN	<pre>SELECT p.productCode, p.productName, od.orderNumber FROM products AS p LEFT JOIN orderdetails AS od ON p.productCode = od.productCode;</pre>	Joins <code>products</code> and <code>orderdetails</code> tables, returning all products and their orders (if any).
RIGHT JOIN	<pre>SELECT e.employeeNumber, e.lastName, o.officeCode FROM offices AS o RIGHT JOIN employees AS e ON o.officeCode = e.officeCode;</pre>	Joins <code>offices</code> and <code>employees</code> tables, returning all employees and their offices (if any).

Clause	How to use	Explained
CROSS JOIN	<pre>SELECT p.productName, pl.textDescription FROM products AS p CROSS JOIN productlines AS pl;</pre>	Returns all possible combinations of products and product line descriptions.
JOIN MULTIPLE	<pre>SELECT o.orderNumber, c.customerName, p.productName FROM orders AS o JOIN customers AS c ON o.customerNumber = c.customerNumber JOIN orderdetails AS od ON o.orderNumber = od.orderNumber JOIN products p ON od.productCode = p.productCode;</pre>	Joins four tables: <code>orders</code> , <code>customers</code> , <code>orderdetails</code> , and <code>products</code> .
JOIN SELF	<pre>SELECT e1.firstName ' ' e1.lastName AS employee, e2.firstName ' ' e2.lastName AS manager FROM employees AS e1 LEFT JOIN employees AS e2 ON e1.reportsTo = e2.employeeNumber;</pre>	Self-join example listing employees and their respective managers.

Subqueries

Clause	How to use	Explained
SUBQUERY IN SELECT	<pre>SELECT productName, buyPrice, (SELECT AVG(buyPrice) FROM products) AS avg_price FROM products;</pre>	Includes a subquery that calculates the average price for all products.
SUBQUERY IN FROM	<pre>SELECT productLine, avg_price FROM (SELECT productLine, AVG(buyPrice) AS avg_price FROM products GROUP BY productLine) AS line_averages WHERE avg_price > 100;</pre>	Finds product lines with an average price greater than 100 using a subquery.
SUBQUERY IN WHERE	<pre>SELECT productName, buyPrice FROM products p1 WHERE p1.buyPrice > (SELECT AVG(p2.buyPrice) FROM products p2 WHERE p1.productLine = p2.productLine) ORDER BY productLine, buyPrice DESC;</pre>	This query selects products that are more expensive than the average price in their respective product line, ordered by product line and price in descending order.

Clause	How to use	Explained
SUBQUERY WITH IN	<pre>SELECT productName, buyPrice FROM products WHERE productCode IN (SELECT productCode FROM orderdetails WHERE orderNumber = 10100);</pre>	Finds products that were ordered in order 10100.
SUBQUERY WITH EXISTS	<pre>SELECT customerName FROM customers c WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customerNumber = c.customerNumber AND o.orderDate >= '2023-01-01');</pre>	This query retrieves the names of customers who have placed at least one order on or after January 1, 2023.

Subqueries

Clause

How to use

=

```
SELECT orderNumber,
       orderDate,
       totalAmount
FROM orders
WHERE customerNumber = (
    SELECT customerNumber
    FROM customers
    WHERE customerName = 'Mini Gifts
                        Distributors Ltd.'
)
ORDER BY orderDate DESC;
```

Explained

This query selects all orders for a specific customer named 'Mini Gifts Distributors Ltd.', ordered by date from most recent to oldest.

CTE

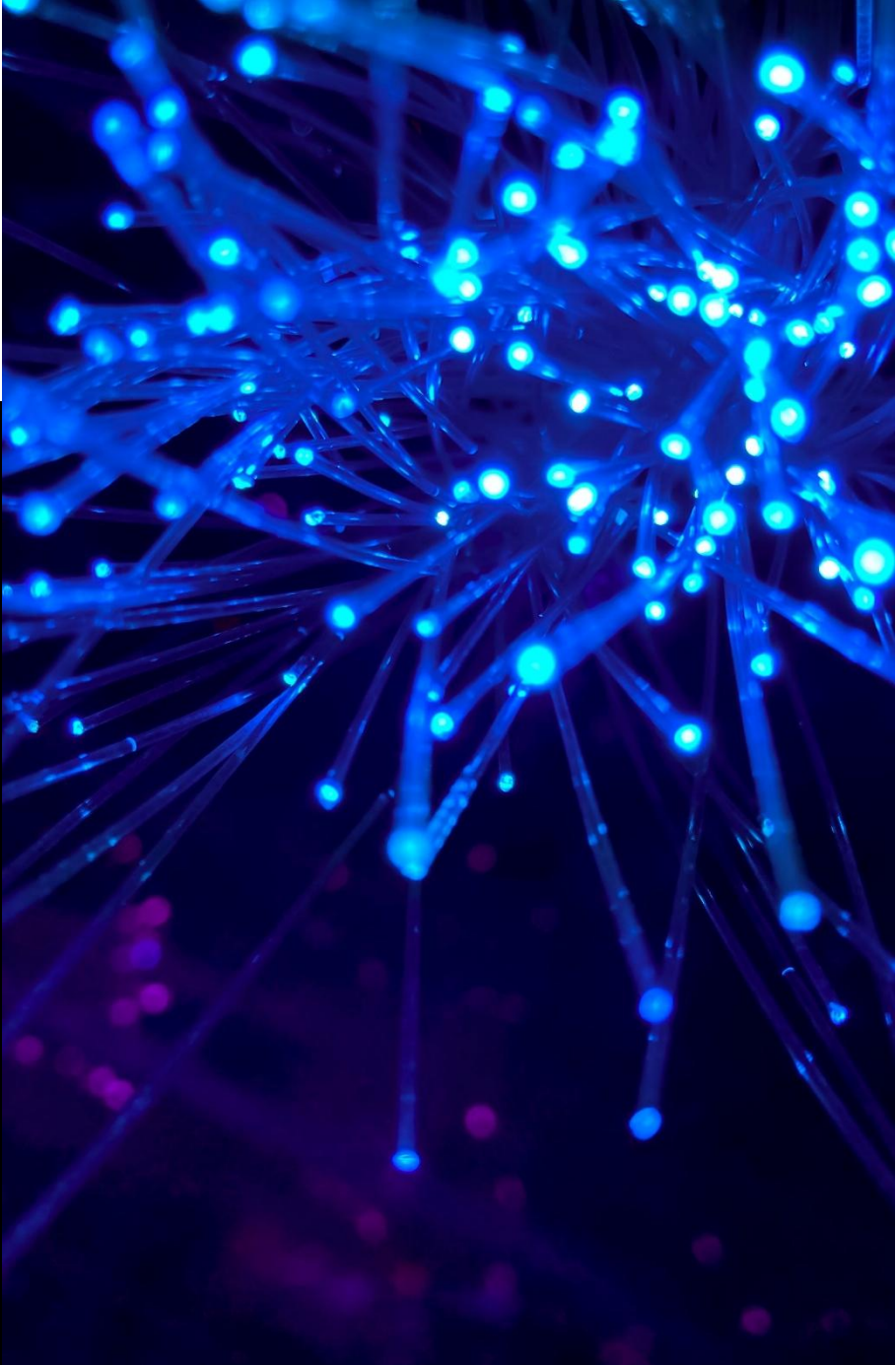
```
WITH order_totals AS (
    SELECT orderNumber,
           SUM(quantityOrdered * priceEach)
           AS total_amount
    FROM orderdetails
    GROUP BY orderNumber
)
SELECT o.orderNumber,
       o.orderDate,
       ot.total_amount
FROM orders o
JOIN order_totals ot
    ON o.orderNumber = ot.orderNumber
ORDER BY ot.total_amount DESC;
```

This query calculates the total amount for each order using a CTE and then joins the orders table with the CTE to display order details with total amounts, ordered by total amount in descending order.

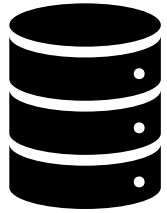
Common Table Expressions (CTEs) in SQL: for improved readability and organization of complex queries.

A CTE is a temporary result set that you can reference within a SELECT, INSERT, UPDATE, or DELETE statement.

They are defined using the WITH keyword, followed by a query that generates the CTE.



SQL: crear, eliminar e insertar



Entorno 1

Método 1: Drop + Create + Insert

Método 2: Merge + Update → Delta



Entorno 2

DROP TABLE IF EXISTS Nombre_De_La_Tabla;

CREATE TABLE Nombre_De_La_Tabla (
[edad] INT,
[salario] FLOAT,
[nombre] NVARCHAR(255));

INSERT INTO Nombre_De_La_Tabla VALUES (?, ?, ?);

Método 1

Método 2

MERGE INTO {NEW_TABLE_NAME} AS target USING

Si un registro ya existe y tiene cambios, se actualiza con UPDATE. Si no existe, se inserta con INSERT

El bloque `with conn_local.cursor() as cursor:` inicia un contexto que gestiona automáticamente el cursor para ejecutar comandos SQL en la base de datos.

```
# Conectar a SQL Server Local
print(f"Conectando a SQL Server Local...")
conn_local = pyodbc.connect(local_conn_str)

with conn_local.cursor() as cursor:
    # Eliminar la tabla si ya existe
    drop_table_sql = f"DROP TABLE IF EXISTS
{NEW_TABLE_NAME}"
    cursor.execute(drop_table_sql)
    conn_local.commit()
    print(f"    - Tabla eliminada si existía.")
```

Objetivo: Si la tabla ya existe, la elimina para evitar conflictos al recrearla.

DROP TABLE IF EXISTS: Comando SQL para eliminar la tabla si existe.

`conn_local.commit():` Confirma los cambios en la base de datos.

El código establece una conexión con SQL Server Local utilizando `pyodbc`, lo que permite interactuar con la base de datos. Luego, se crea un cursor, que es un intermediario para ejecutar comandos SQL. A continuación, se ejecuta una consulta `DROP TABLE IF EXISTS`, que elimina la tabla si ya existe, asegurando que no haya conflictos al recrearla. Una vez ejecutada la consulta, se usa `commit()` para confirmar los cambios en la base de datos. Finalmente, se imprime un mensaje indicando que la tabla ha sido eliminada, lo que facilita el seguimiento del proceso en la ejecución.

```
# Crear la tabla en SQL Server Local con tipos de datos ajustados
create_table_sql = f"""
CREATE TABLE {NEW_TABLE_NAME} (
    {', '.join([
        f'[{col}] FLOAT' if df[col].dtype == np.float32
        else f'[{col}] INT' if df[col].dtype == np.int32
        else f'[{col}] NVARCHAR(255)' for col in df.columns
    ])}
);
"""

cursor.execute(create_table_sql)
conn_local.commit()
print(f" Tabla {NEW_TABLE_NAME} creada correctamente en SQL
Server Local.")
```

Objetivo: Crear una tabla nueva en SQL Server con columnas basadas en los tipos de datos del DataFrame (df).

El código convierte automáticamente las columnas de Pandas a tipos de SQL Server: np.float32 → FLOAT, np.int32 → INT, Cualquier otro tipo → NVARCHAR(255).

Con esto se evita errores de carga en la ETL.

El código crea una nueva tabla en SQL Server Local, asegurando que los tipos de datos sean adecuados según las columnas del DataFrame. Para ello, se genera dinámicamente una sentencia CREATE TABLE, donde las columnas numéricas se asignan como FLOAT o INT, mientras que las de texto se almacenan como NVARCHAR(255). Luego, la consulta se ejecuta a través del cursor y se utiliza commit() para confirmar los cambios en la base de datos.

```
# Insertar los datos en SQL Server Local
placeholders = ', '.join(['?' for _ in df.columns])
insert_sql = f"INSERT INTO {NEW_TABLE_NAME} VALUES
({placeholders})"

cursor.fast_executemany = True
cursor.executemany(insert_sql, df.values.tolist())
conn_local.commit()

print(f" {df.shape[0]} filas insertadas en
{NEW_TABLE_NAME}.")
```

Objetivo: Insertar los datos del DataFrame (df) en la tabla recién creada.

- placeholders: Crea una lista de signos ? que se usan como placeholders en SQL (uno por cada columna).
- fast_executemany = True: Optimiza la inserción en lotes para mejorar el rendimiento en SQL Server.
- cursor.executemany(insert_sql, df.values.tolist()): Inserta todas las filas del DataFrame en una sola operación.

El código inserta los datos del DataFrame en la tabla recién creada en SQL Server Local. Primero, genera una consulta INSERT INTO con marcadores de posición (?), que permiten insertar valores de manera segura y eficiente. Luego, habilita fast_executemany para optimizar la inserción masiva de datos. Con executemany(), todas las filas del DataFrame se envían en una sola operación para mejorar el rendimiento. Finalmente, se confirma la transacción con commit() y se imprime un mensaje indicando cuántas filas fueron insertadas, facilitando el monitoreo del proceso.

```
# Usar MERGE para actualizar e insertar datos
columns = ', '.join([f'[{col}]' for col in df.columns])
placeholders = ', '.join(['?' for _ in df.columns])

merge_sql = f"""
MERGE INTO {NEW_TABLE_NAME} AS target
USING (SELECT {columns} FROM (VALUES ({placeholders})) AS temp({columns})) AS source
ON target.[id] = source.[id] -- Reemplaza 'id' con la clave primaria real

WHEN MATCHED AND (
    { ' OR '.join([f'target.[{col}] <> source.[{col}]' for col in df.columns if col != 'id']) }
) THEN
    UPDATE SET
    { ', '.join([f'target.[{col}] = source.[{col}]' for col in df.columns if col != 'id']) }

WHEN NOT MATCHED THEN
    INSERT ({columns}) VALUES ({placeholders});
"""

cursor.fast_executemany = True
cursor.executemany(merge_sql, df.values.tolist())
conn_local.commit()

print(f" {df.shape[0]} filas actualizadas o insertadas en {NEW_TABLE_NAME}.")
```

El código usa la sentencia MERGE de SQL Server para realizar una inserción y actualización masiva de datos de manera eficiente. Primero, obtiene los nombres de las columnas y genera los marcadores de posición (?) para insertar los valores de manera segura. Luego, construye la consulta MERGE, que compara la tabla existente con los datos nuevos mediante la clave primaria (id). Si un registro ya existe y tiene cambios, se actualiza con UPDATE. Si no existe, se inserta con INSERT. Finalmente, fast_executemany optimiza la ejecución para mejorar la velocidad en la carga de grandes volúmenes de datos.

Comentarios y preguntas